

Algorithm Design — Lab Assignment 4

Convex Hull: Monotone Chain vs. Divide and Conquer

Student ID: 411321344

Name: Wilkend AUGUSTE

Course: Lab of Algorithm

Date: Wednesday 1, 2026



National Dong Hwa University

Department of Computer Science and Information Engineering

1. Problem Definition

Given a finite set of points $P = \{p_1, p_2, \dots, p_n\}$ in the 2D plane, the goal of the convex hull problem is to find the smallest convex polygon that encloses all points in P . This polygon is called the convex hull of P .

More formally, a point p is on the convex hull if it cannot be expressed as a convex combination of any other points in P . That means:

$$CH(P) = \{p \in P \mid p \neq \sum \lambda_i \cdot p_i, \text{ where } \lambda_i \geq 0 \text{ and } \sum \lambda_i = 1\}$$

In simpler terms, if you were to stretch a rubber band around all the points and let it snap tight, the boundary it forms is the convex hull. Only the outermost points, the ones that form corners or edges of the hull are included as vertices.

This problem appears in many real-world scenarios. For example, in robotics path planning, the convex hull is used to define the boundary of an obstacle. In computer graphics, it helps determine bounding shapes. In data analysis, it identifies outliers or boundary points of a dataset. In geographic information systems (GIS), it outlines the region covered by a set of GPS coordinates.

In this lab, I study and compare two classical algorithms for computing the convex hull: Monotone Chain also called Andrew's monotone chain algorithm and Divide and Conquer. Both algorithms are efficient, but they have different strategies and trade-offs that are worth analyzing.

Figure 1: Convex Hull of a Point Set

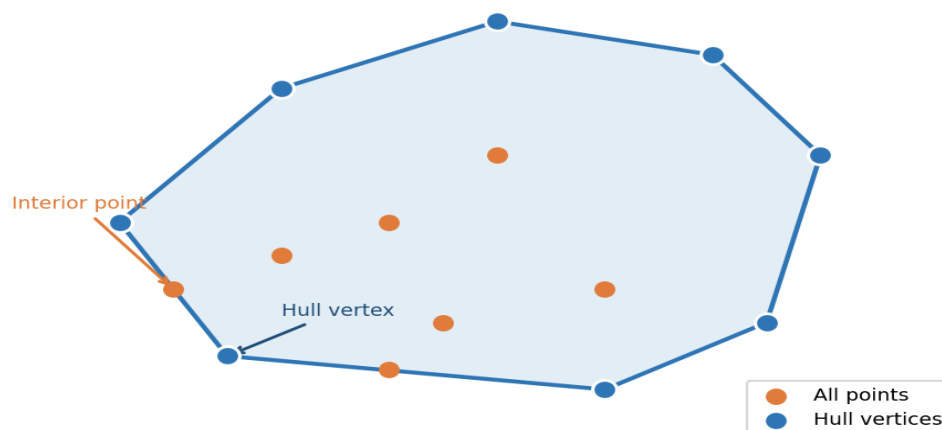


Figure 1: Example of a convex hull surrounding a point set

2. Monotone Chain Algorithm

2.1 Overview

The Monotone Chain algorithm, introduced by Andrew in 1979, is one of the most practical and elegant methods for computing convex hulls. The key insight is that if you sort the points by x-coordinate and by y when x values are tied, you can construct the hull in two passes, once for the lower boundary and once for the upper boundary.

The algorithm uses a stack-based approach. For each pass, you scan through the sorted points and maintain a stack. If adding a new point would make a right turn (clockwise direction) with the last two points on the stack, those points are not on the hull and get popped off. This relies on the cross product to detect turn direction.

2.2 Cross Product and Turn Detection

Given three points A, B, and C, the cross product of vectors AB and AC is:

$$\text{cross}(A, B, C) = (B.x - A.x) * (C.y - A.y) - (B.y - A.y) * (C.x - A.x)$$

If $\text{cross}(A, B, C) > 0$, the turn is counterclockwise (left turn, valid for lower hull). If $\text{cross}(A, B, C) < 0$: the turn is clockwise (right turn, pop from stack). If $\text{cross}(A, B, C) = 0$: the three points are collinear.

During the lower hull scan, we keep points only when they make a left turn or are collinear. For the upper hull, we reverse the direction. This is the core filtering mechanism that eliminates interior points.

Figure 3: Cross Product — Turn Test in Monotone Chain

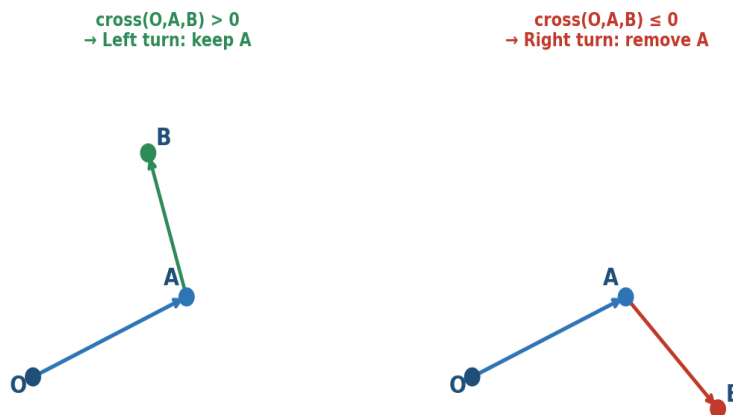


Figure 2: Cross product turn detection — left turn (valid) vs right turn (pop)

2.3 Pseudocode

```

MONOTONE_CHAIN(P):
    Sort P lexicographically by (x, y)
    n = |P|
    lower = []
    for i from 0 to n-1:
        while |lower| >= 2 and cross(lower[-2], lower[-1], P[i]) <= 0:
            lower.pop()
        lower.append(P[i])
    upper = []
    for i from n-1 down to 0:
        while |upper| >= 2 and cross(upper[-2], upper[-1], P[i]) <= 0:

```

```
upper.pop()
upper.append(P[i])
return lower[0..n-2] + upper[0..n-2]
```

Figure 2: Monotone Chain — Step by Step

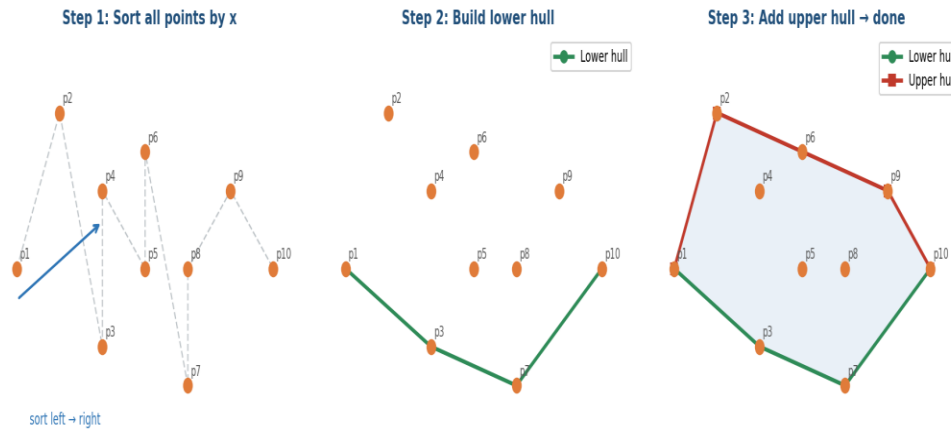


Figure 3: Building the lower and upper hulls in Monotone Chain

2.4 Complexity Analysis

Time Complexity:

- Sorting: $O(n \log n)$. This dominates the total runtime.
- Lower hull scan: $O(n)$. Each point is pushed and popped at most once.
- Upper hull scan: $O(n)$. Same argument, each point is pushed/popped at most once.
- Total: $O(n \log n)$.

Space Complexity: $O(n)$ - the two stacks together hold at most n points in total.

The amortized cost of the while-loop pops is $O(n)$ total, even though the loop might pop several points at once for some iterations. This is because each point can only be pushed and popped once across the entire pass.

3. Divide and Conquer Algorithm

3.1 Overview

The Divide and Conquer (D&C) approach to convex hull follows the classic algorithmic paradigm: split the problem into smaller subproblems, solve them recursively, then merge the solutions. For convex hull, the merge step is the hardest part, combining two convex hulls into one requires finding the upper and lower tangents.

The basic idea is to sort the points by x -coordinate, then split them into a left half and a right half. Recursively compute the convex hull of each half, then merge the two hulls by finding the two tangent lines that connect them, one on top and one on bottom.

3.2 Pseudocode

```
DIVIDE_AND_CONQUER_HULL(P):
    Sort P by x-coordinate
    return HULL_RECURSIVE (P, 0, |P|-1)

HULL_RECURSIVE (P, lo, hi):
    if hi - lo <= 2:
        return BRUTE_FORCE_HULL(P[lo..hi])
    mid = (lo + hi) / 2
    left_hull = HULL_RECURSIVE(P, lo, mid)
    right_hull = HULL_RECURSIVE(P, mid+1, hi)
    return MERGE_HULLS(left_hull, right_hull)

FIND_UPPER_TANGENT (L, R):

    // L = left hull (sorted CCW), R = right hull (sorted CCW)
    // Start from rightmost point of L and leftmost point of R

    a = index of rightmost point in L
    b = index of leftmost point in R
    changed = true
    while changed:
        changed = false
        // Move right pointer up
        while cross(R[b], L[a], L[(a+1) mod |L|]) > 0:
            a = (a + 1) mod |L|
            changed = true
        // Move left pointer up
        while cross(L[a], R[b], R[(b-1+|R|) mod |R|]) < 0:
            b = (b - 1 + |R|) mod |R|
            changed = true
    return (a, b)
    // indices of upper tangent endpoints

FIND_LOWER_TANGENT(L, R):
    // symmetric to upper tangent
    // Similar logic but moving in opposite direction

MERGE_HULLS(L, R):
    (ut_L, ut_R) = FIND_UPPER_TANGENT(L, R)
    (lt_L, lt_R) = FIND_LOWER_TANGENT(L, R)
    merged = []
    // Take upper chain: go from lt_L around L clockwise to ut_L
    i = lt_L
    while i != ut_L:
        merged.append(L[i])
        i = (i + 1) mod |L|
    merged.append(L[ut_L])
    // Take lower chain: go from ut_R around R clockwise to lt_R
    i = ut_R
    while i != lt_R:
        merged.append(R[i])
        i = (i + 1) mod |R|
    merged.append(R[lt_R])
    return merged
```

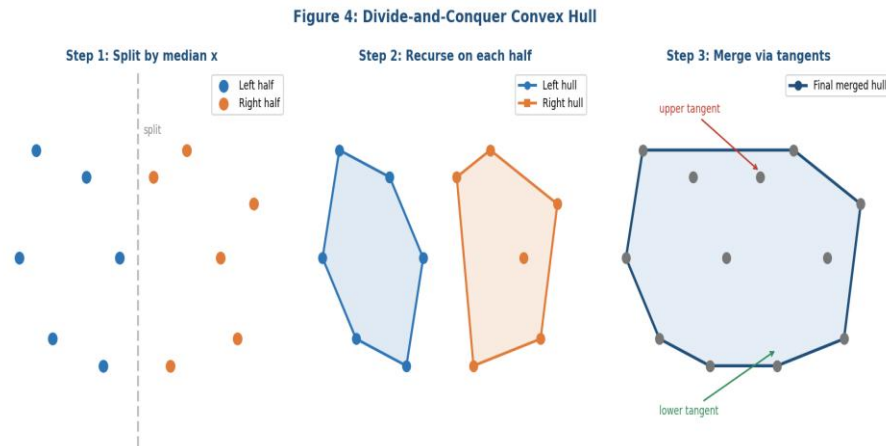


Figure 4: Divide and Conquer — splitting, recursing, and merging with tangents

3.3 Why the Merge Step is Critical

The merge step is the most technically demanding part of the D&C approach. The two sub-hulls are already valid convex polygons individually but combining them correctly requires identifying exactly which points from each hull lie on the outer boundary of the combined hull.

The upper tangent is the line segment that touches the top of both hulls, no points from either hull fall above this line. The lower tangent is symmetric. Together they define the stitching boundary. Finding each tangent requires an iterative adjustment: you move one pointer at a time and alternate between adjusting the left and right index until neither pointer needs to move. This can take $O(n)$ in the worst case for each tangent but typically runs in $O(\log n)$ with careful implementation on convex polygons.

If the merge step is wrong, the result may have indentations or miss outer points entirely, making it non-convex. So, this step requires careful handling of edge cases like collinear points and degenerate configurations.

3.4 Complexity Analysis

The recurrence relation for the D&C approach is:

$$T(n) = 2T(n/2) + M(n)$$

where $M(n)$ is the cost of merging two hulls of total size n . If merging is done naively in $O(n)$, then by the Master Theorem (case 2), $T(n) = O(n \log n)$. However, finding tangents on circular hull arrays can take $O(n)$ per tangent in the worst case, which gives the same bound.

Some implementations achieve $O(\log n)$ tangent-finding using binary search on the sorted convex hull vertices, which reduces the merge cost and gives $T(n) = O(n \log^2 n)$ total, a bit worse than Monotone Chain in theory but acceptable in practice.

Space: $O(n \log n)$ due to the recursion stack and intermediate hull arrays at each recursive level.

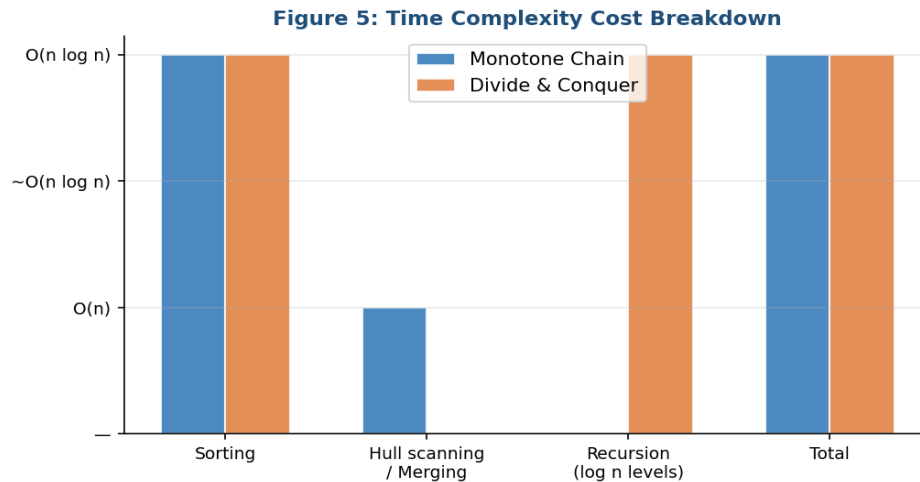


Figure 5: Complexity comparison between the two algorithms

4. Comparison and Discussion

4.1 Side-by-Side Comparison

Aspect	Monotone Chain	Divide & Conquer
Sorting step	$O(n \log n)$ required	Not needed (median split)
Build strategy	Scan all points once	Recursive divide-and-conquer
Merge step	Not needed (single pass)	Complex upper/lower tangent finding
Time complexity	$O(n \log n)$ overall	$O(n \log^2 n)$ worst case
Space complexity	$O(n)$ for stacks	$O(n)$ call stack recursion
Code complexity	Simple, easy to implement	More complex merge logic
Degeneracy handling	Handles collinear naturally	Must handle carefully in merge
Best use case	Large datasets, simplicity	Parallel/distributed settings

4.2 Discussion

Looking at the comparison table above, both algorithms have their strengths, but they are suited for different situations. Here is my analysis of each:

Monotone Chain is generally better for most practical applications. The algorithm is straightforward to implement correctly, sorting plus two stack-based scans. There are no tricky edge cases in the merge step because there is no merge step. The $O(n \log n)$ time is guaranteed, and the constant factors are small. I would personally use this algorithm in competitive programming or production code.

Divide and Conquer is more interesting from a theoretical standpoint. The recursive structure makes it a good teaching example of the D&C paradigm. It also has a natural parallelization opportunity; both halves can be computed on separate processors simultaneously. In a distributed computing environment, this could lead to real speedups even if the sequential complexity is similar or slightly worse.

4.3 Strengths and Limitations

Monotone Chain Strengths:

- Very simple code, easy to debug and verify
- Exactly $O(n \log n)$ runtime, dominated entirely by the sort
- Low constant factors, extremely fast in practice
- Handles collinear points easily (can include or exclude with a minor change)

Monotone Chain Limitations:

- Inherently sequential, the second pass depends on the first
- Not naturally parallelizable across distributed systems
- All points must be in memory at once for sorting

Divide and Conquer Strengths:

- Naturally recursive and parallelizable, left and right halves are independent
- Good for divide-and-conquer frameworks or parallel computing
- The recursive structure allows incremental hull updates if a new batch of points arrives on one side only

Divide and Conquer Limitations:

- The merge step with tangent-finding is complex to implement correctly
- More memory overhead due to recursive calls and intermediate hull storage
- Harder to handle degenerate cases (collinear points, duplicate x-values)
- In practice, often slower than Monotone Chain due to overhead

4.4 Real-World Use Cases

The choice between these two algorithms really comes down to the use case. In robotics and real-time applications where you need a fast, reliable result, Monotone Chain is the clear winner. In distributed systems or GPU computation where parallelism is available, the D&C structure might give better throughput even if the per-unit complexity is slightly higher.

For this lab, if I had to choose one algorithm to implement in production, I would go with Monotone Chain. It is simpler, faster, and less error prone. But studying the Divide and Conquer approach gave me a deeper understanding of the merge tangent concept, which is used in other geometric algorithms too.

5. Conclusion

In this lab, I studied two classical algorithms for computing the convex hull of a 2D point set: Monotone Chain and Divide and Conquer.

Monotone Chain works by sorting the points lexicographically and then doing two linear scans, one for the lower hull and one for the upper hull using cross products to remove interior points. It achieves $O(n \log n)$ time complexity and is simple to implement correctly.

Divide and Conquer splits the sorted point set recursively into halves, computes each hull independently, and then merges them by finding upper and lower tangent lines. The merge step is the key challenge and requires careful iterative pointer adjustment. The time complexity is $O(n \log n)$ or $O(n \log^2 n)$ depending on the tangent-finding strategy.

Both algorithms are correct and efficient, but Monotone Chain is generally preferred in practice due to its simplicity and lower constant factors. Divide and Conquer is better in parallel or distributed settings where the independence of subproblems can be exploited.

This assignment helped me understand how geometric algorithms use cross products for turn detection, how recursive decomposition applies to spatial problems, and why the merge step in D&C requires such careful attention to detail.